

GUÍA DE LABORATORIO TÉCNICO: INTRODUCCIÓN A PYTHON Y ANÁLISIS DE MICRODATOS

Institución: Universidad Nacional Agraria La Molina (UNALM)
Facultad: Economía y Planificación
Instructor: Lic. Willy Olivos Apéstegui
Entorno de Ejecución: Google Colab (colab.research.google.com)

1. ENTORNO DETERMINISTA VS. ENTORNO COGNITIVO

El analista de comercio internacional moderno no puede depender exclusivamente de herramientas tradicionales como Excel cuando se enfrenta a registros transaccionales masivos del Estado (SUNAT). Este laboratorio establece las bases para migrar hacia un control determinista utilizando código puro (Python), garantizando la reproducibilidad matemática, la limpieza sistemática de anomalías y la posterior explotación visual mediante Inteligencia Artificial.

Herramienta	Volumen Operativo	Ventaja Principal	Limitación Crítica
Excel / Power BI	Bajo a Medio	Interfaz visual intuitiva.	Colapso de memoria con archivos corruptos o millones de filas.
Python (Google Colab)	Masivo (Millones de registros)	Control total del dato, reproducibilidad y	Requiere curva de aprendizaje en sintaxis lógica.

Herramienta	Volumen Operativo	Ventaja Principal	Limitación Crítica
		automatización absoluta.	
IA Generativa (Canvas)	Variable (Prefiere síntesis)	Velocidad gerencial e interpretación lingüística.	Riesgo latente de alucinación matemática si los datos no están limpios.

2. EJERCICIO 1: MI PRIMER COMANDO (HOLA MUNDO)

La función nativa `print()` ordena al intérprete de Python desplegar un bloque de texto o el resultado de un cálculo en la pantalla. Todo texto plano (cadena de caracteres) debe ir obligatoriamente delimitado por comillas.

Instrucción para el Alumno: Abre un nuevo cuaderno en Google Colab con tu cuenta institucional, copia el siguiente bloque de código en la primera celda y ejecútalo presionando **Shift + Enter**.

```
# EJERCICIO 1: Validación del Entorno de Ejecución
print("Hola UNALM - Bienvenidos al Laboratorio de Automatización de Datos")
```

3. CONFIGURACIÓN DEL ECOSISTEMA: VARIABLES Y BIBLIOTECAS

- **Variables:** Estructuras lógicas que funcionan como contenedores digitales con un nombre asignado, capaces de almacenar desde un número simple hasta matrices

complejas de datos transaccionales.

- **Bibliotecas (Libraries):** ** Extensiones modulares especializadas que integran funciones avanzadas para evitar programar algoritmos matemáticos desde cero.

En este laboratorio inicial invocaremos las dos herramientas estándar de la industria de datos:

1. **Pandas (alias pd):** Diseñada específicamente para la manipulación, estructuración y limpieza de DataFrames (tablas organizadas en filas y columnas).
2. **Matplotlib (alias plt):** El motor fundamental de renderizado para la construcción de gráficos estadísticos bidimensionales con precisión de píxel.

Nota Técnica del Servidor: Al ejecutar el taller sobre la infraestructura en la nube de Google Colab, las bibliotecas pandas y matplotlib ya se encuentran instaladas por defecto. No se requiere ejecutar comandos de instalación remota.

4. EJERCICIO 2: CONSTRUCCIÓN DE UN DATASET Y GRÁFICO BASADO EN LENGUAJE NATURAL

A continuación, estructuraremos un DataFrame manual que simula las toneladas métricas exportadas de Palta Hass durante la ventana comercial más activa del año en el Perú. Posteriormente, usaremos el motor de visualización para graficar el comportamiento temporal.

```
# EJERCICIO 2: Simulación y Graficación de Volumen de Palta Hass
import pandas as pd
import matplotlib.pyplot as plt
```

```
# 1. Definición del diccionario base con vectores de datos
datos_palta = {
    'Mes': ['Abril', 'Mayo', 'Junio', 'Julio'],
    'Toneladas': [12000, 18500, 24000, 15100]
}
```

```
# 2. Conversión estructural del diccionario a un DataFrame de Pandas
df_palta = pd.DataFrame(datos_palta)
```

```
# 3. Invocación del método gráfico integrado en Pandas (Gráfico de Barras)
```

```
df_palta.plot(x='Mes', y='Toneladas', kind='bar', color='darkgreen',
legend=False)
```

```
# 4. Inyección de directrices de personalización y rotulado
estadístico
plt.title('Campaña Palta Hass: Volumen de Exportación Mensual',
weight='bold', fontsize=14, pad=15)
plt.xlabel('Ventana Temporal de Análisis', fontsize=11, labelpad=10)
plt.ylabel('Volumen Total (Toneladas Métricas)', fontsize=11,
labelpad=10)
plt.xticks(rotation=0) # Fuerza la orientación horizontal de los
textos del eje X
plt.grid(axis='y', linestyle='--', alpha=0.5) # Inserta líneas guía
con opacidad atenuada

# 5. Orden de renderizado final en la interfaz del usuario
plt.show()
```

5. PROTOCOLO OPERATIVO: INGESTA GENERALIZADA DE MICRODATOS ADUANEROS (.DBF)

El verdadero valor de la automatización radica en construir scripts reproducibles. Los alumnos utilizarán un archivo binario transaccional relacional duro (extensión .DBF) extraído de los servidores oficiales de la SUNAT.

La ventaja crítica de este script es que la arquitectura matemática interna permanece idéntica. Si el analista comercial desea cambiar de mercado de estudio (por ejemplo, evaluar la dinámica de las Uvas Frescas o los Arándanos), ****solo debe modificar una sola línea de código**** especificando el nombre exacto de la base de datos descargada.

```
# PROTOCOLO DE CONVERSIÓN AUTOMATIZADA DE ADUANAS (.DBF)
# Requiere instalación previa de la biblioteca dbfread para procesar
binarios dBase
!pip install dbfread --quiet

from dbfread import DBF
```

```

import pandas as pd
import matplotlib.pyplot as plt

#
=====
=====
# CAMBIO OPERATIVO CRÍTICO: Modifica únicamente el nombre del
archivo entre comillas
#
=====
=====
archivo_sunat = "00282737.DBF" # Reemplazar aquí por "arándano.DBF"
o "uva.DBF" según el caso

# 1. Ingesta del archivo binario ignorando errores de codificación
regional
tabla_dbf = DBF(archivo_sunat, encoding='latin1',
char_decode_errors='ignore')

# 2. Transformación fluida de la estructura binaria a DataFrame de
Pandas
df_aduanas = pd.DataFrame(iter(tabla_dbf))

# 3. Verificación de integridad estructural (Imprime el número total
de filas y columnas cargadas)
print("PROCESAMIENTO EXITOSO.")
print(f"La base de datos actual contiene {df_aduanas.shape[0]}
registros transaccionales activos.")

# A partir de aquí, el DataFrame 'df_aduanas' queda disponible en
memoria
# para aplicar las técnicas de limpieza, extracción de oligopolios e
interpretación visual.

```